

An Analysis of Adversarial examples

Adversarial examples

Adversarial examples -- Part 1

Data poisoning Poisoning consists of contaminating the training dataset with data designed to increase errors in the output. Given that learning algorithms are shaped by their training datasets, poisoning can effectively reprogram algorithms with potentially malicious intent. Concerns have been raised especially for user-generated training data, e.g. for content recommendation or natural language models. The ubiquity of fake accounts offers many opportunities for poisoning. Facebook reportedly removes around 7 billion fake accounts per year. Poisoning has been reported as the leading concern for industrial applications. On social medias, disinformation campaigns attempt to bias recommendation and moderation algorithms, to push certain content over others. A particular case of data poisoning is the backdoor attack, which aims to teach a specific behavior for inputs with a given trigger, e.g. a small defect on images, sounds, videos or texts.

Adversarial examples -- Part 2

Class imbalance In realistic deployment environments, the proportion of malicious samples can be extremely low, ranging from 0.01% to 2% of total data. This unbalanced distribution causes models to develop a bias towards the majority class, achieving high accuracy but failing to identify malicious samples. Prior approaches to this problem have included both data-level solutions and sequence-specific models. Methods like n-gram and Long Short-Term Memory (LSTM) networks can model sequential data, but their performance has been shown to decline significantly when malware samples are realistically proportioned in the training set, demonstrating the limitations in realistic security contexts. To address this issue, one approach has been to adapt models from natural language processing, such as BERT. This method involves treating sequences of application activities as a form of "language" and fine-tuning a pre-trained BERT model on the specific task. A study applying this technique to Android activity sequences reported an F1 score of 0.919 on a dataset with only 0.5% malware samples. This result was a significant improvement over LSTM and n-gram models, demonstrating the potential of pre-trained models to handle class imbalance in malware detection.

Adversarial examples -- Part 3

Iterate below
Boundary search
Gradient update
Compute the gradient
Find the step size
Boundary search uses a modified binary search to find the point in which the boundary (as defined by S) intersects with the line between x and x' . The next step involves calculating the gradient for x , and update the

original x using this gradient and a pre-chosen step size. HopSkipJump authors prove that this iterative algorithm will converge, leading x to a point right along the boundary that is very close in distance to the original image. However, since HopSkipJump is a proposed black box attack and the iterative algorithm above requires the calculation of a gradient in the second iterative step (which black box attacks do not have access to), the authors propose a solution to gradient calculation that requires only the model's output predictions alone. By generating many random vectors in all directions, denoted as u_b , an approximation of the gradient can be calculated using the average of these random vectors weighted by the sign of the boundary function on the image $x' + \delta u_b$, where δ is the size of the random vector perturbation:

Adversarial examples -- Part 4

Threat modeling – formalize the attackers goals and capabilities with respect to the target system.
Attack simulation – formalize the optimization problem the attacker tries to solve according to possible attack strategies.
Attack impact evaluation
Countermeasure design
Noise detection (for evasion-based attack)
Information laundering – Alter the information received by adversaries (for model stealing attacks)

Adversarial examples -- Part 5

Fast gradient sign method One of the first proposed attacks for generating adversarial examples was proposed by Google researchers Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. The attack was called fast gradient sign method (FGSM), and it consists of adding a linear amount of in-perceivable noise to the image and causing a model to incorrectly classify it. This noise is calculated by multiplying the sign of the gradient with respect to the image we want to perturb by a small constant epsilon. As epsilon increases, the model is more likely to be fooled, but the perturbations become easier to identify as well. Shown below is the equation to generate an adversarial example where x is the original image, ϵ is a very small number, Δ_x is the gradient function, J is the loss function, θ is the model weights, and y is the true label.

Adversarial examples -- Part 6

Square attack Square attacks were introduced in 2020 as a black-box evasion adversarial attack technique based on querying classification scores without the need for gradient information.

An Analysis of Adversarial examples

Adversarial examples

As a score based black box attack, this adversarial approach is able to query probability distributions across model output classes, but has no other access to the model itself. According to the paper's authors, the proposed Square Attack required fewer queries than when compared to state-of-the-art score-based black box attacks at the time. To describe the function objective, the attack defines the classifier as $f: [0, 1]^d \rightarrow \mathbb{R}^K$, with d representing the dimensions of the input and K as the total number of output classes. $f_k(x)$ returns the score (or a probability between 0 and 1) that the input x belongs to class k , which allows the classifier's class output for any input x to be defined as $\arg\max_{k=1, \dots, K} f_k(x)$. The goal of this attack is as follows:

Adversarial examples -- Part 7

In other words, finding some perturbed adversarial example $\hat{x}$ such that the classifier incorrectly classifies it to some other class under the constraint that $\hat{x}$ and x are similar. The paper then defines loss L as $L(f(\hat{x}), y) = f_y(\hat{x}) - \max_{k \neq y} f_k(\hat{x})$ and proposes the solution to finding adversarial example $\hat{x}$ as solving the below constrained optimization problem: